you'll see in the next example, is accomplished through reapplying the functions you used to create the graphical elements based on your data.

Each `.enter()` or `.exit()` event can include actions taken on child elements. This is mostly useful with `.enter()` events, where you use the `.append()` function to add new elements. If you declare this new appended element as a variable, and if that element is amenable to child elements, like a `<g>` element is, then you can include any number of child elements. In the case of SVG elements, only `<svg>`, `<g>`, and `<text>` can have child elements, but if you're using D3 with traditional DOM manipulation, then you can use this method to add `<p>` elements to `<div>` elements and so on.

For example, let's say we want to show a bar chart based on our newly measured impact score, and we want the bars on the bar chart to have labels. We need to append `<g>` elements, and not shapes, to the `<svg>` canvas in our initial selection. Because the data is bound to these elements, we can use the same syntax when we add child elements. Because we're using `<g>` elements, we need to set the position using the `transform` attribute. We add child elements to the `.append()` function, and we need to declare it as a variable `tweetG`. This allows `tweetG` to stand in for `d3.select("svg").selectAll("g")` so we don't have to retype it throughout the example. The following listing uses all the same scales to determine size and position as the previous example.

Listing 2.6 Creating labels on `<g>` elements

```
var tweetG = d3.select("svg")
  .selectAll("g")
  .data(incomingData)
  .enter()
  .append("g")
  .attr("transform", function(d) {
  return "translate(" +
    timeRamp(d.tweetTime) + "," + (480 - yScale(d.impact))
    + ")";
  });

tweetG.append("circle")
  .attr("r", function(d) {return radiusScale(d.impact);})
  .style("fill", "#990000")
  .style("stroke", "black")
  .style("stroke-width", "1px");

tweetG.append("text")
  .text(function(d) {return d.user + "-" + d.tweetTime.getHours();});
```

<g> requires a transform, which takes a constructed string.

Uses .getHours() to make the label a bit more legible

In figure 2.25 you can see the result of our code, along with some annotation. The same circles in the same position show that `translate` works much like changing cx and cy for circles, but now we can add other SVG elements, like `<text>` for labels.

The labels are illegible in the bottom left, but they're not much better for the rest. Later on, you'll learn how to make better labels. The inline functions such as `.text(function(d) {return d.user + "-" + d.tweetTime.getHours()})` set the label